

Controle Financeiro Pessoal

Arquitetura de Microsservicos com Spring Boot

Guia Completo para Entrevista Tecnica

Desenvolvedor: Tacio Lucio

GitHub: github.com/Taciolb

Objetivo: Vaga de Desenvolvedor no Mercado Livre

Stack Tecnologico

Spring Boot 3.5.15 | Java 17 | Spring Cloud Gateway | Spring Cloud OpenFeign
JWT (JJWT 0.12.6) | BCrypt | Flyway | PostgreSQL | Hibernate/JPA
Lombok | SpringDoc/Swagger | Docker | Docker Compose | GitHub Actions

1. Visao Geral do Projeto

O Controle Financeiro Pessoal e um sistema completo de gestao financeira construido com arquitetura de microsservicos. O projeto foi desenvolvido como portfolio tecnico para demonstrar dominio de tecnologias enterprise utilizadas em grandes empresas como o Mercado Livre.

O sistema permite que usuarios cadastrem lancamentos financeiros (receitas e despesas), organizem por centros de custo, visualizem o fluxo de caixa e consultem um dashboard agregado com todas as informacoes em uma unica chamada.

Microsservicos do Sistema

Servico	Porta	Banco	Status
api-gateway	8080	—	Concluido
ms-usuarios	8081	CF_usuarios	Concluido
ms-centro-custo	8082	CF_centrocusto	Concluido
ms-lancamentos	8083	CF_lancamentos	Concluido
ms-fluxo-caixa	8084	— (sem banco)	Concluido
bff-financeiro	8085	— (sem banco)	Concluido
frontend Angular	4200	—	Pendente

Fluxo de uma Requisicao

Cliente (Postman / Angular) -> api-gateway:8080 -> valida JWT -> roteia para o servico correto
Exemplo: POST /auth/login -> ms-usuarios:8081 (rota publica, sem JWT) Exemplo: GET /lancamentos
-> ms-lancamentos:8083 (exige JWT valido) Exemplo: GET /dashboard -> bff-financeiro:8085 -> chama
3 microsservicos simultaneamente

2. Arquitetura de Microsservicos

A arquitetura de microsservicos divide o sistema em servicos independentes, cada um com sua propria responsabilidade, banco de dados e ciclo de deploy. Essa abordagem e amplamente utilizada no Mercado Livre para escalar partes especificas do sistema sem afetar as demais.

Principios aplicados

- **Single Responsibility:** Cada microsservico faz apenas uma coisa. ms-usuarios so gerencia usuarios, ms-lancamentos so gerencia lancamentos.
- **Database per Service:** Cada servico tem seu proprio banco PostgreSQL (CF_usuarios, CF_centrocosto, CF_lancamentos). Nao ha banco compartilhado.
- **API Gateway:** Ponto unico de entrada. Valida JWT antes de rotear para os servicos internos.
- **BFF (Backend for Frontend):** Agrega dados de 3 microsservicos em uma unica resposta para o frontend.
- **Comunicacao via HTTP (Feign):** Microsservicos se comunicam via REST usando Spring Cloud OpenFeign.
- **JWT sem estado (Stateless):** Nao ha sessao. Cada requisicao carrega o token JWT que e validado independentemente em cada servico.

Por que nao um monolito?

Pergunta classica de entrevista: 'Por que voce escolheu microsservicos?' Resposta: Em sistemas financeiros, escalabilidade e isolamento de falhas sao criticos. Com microsservicos, posso escalar so o ms-lancamentos se ele estiver sobrecarregado, sem mexer nos outros. Se o ms-centro-custo cair, o login e os lancamentos continuam funcionando. Alem disso, times diferentes podem trabalhar em servicos diferentes sem conflitos de merge.

3. ms-usuarios (porta 8081)

Responsavel por cadastro, autenticacao e gestao de usuarios. E o unico servico que emite tokens JWT. Os demais servicos apenas validam o token.

Funcionalidades

- POST /auth/registrar — cadastra novo usuario (senha criptografada com BCrypt)
- POST /auth/login — autentica e retorna token JWT
- GET /usuarios — lista todos os usuarios (requer JWT)
- GET /usuarios/{id} — busca usuario por ID
- PUT /usuarios/{id} — atualiza dados do usuario
- DELETE /usuarios/{id} — soft delete (marca ativo=false, nao apaga do banco)

Seguranca e JWT

A senha nunca e salva em texto puro. O BCrypt aplica um hash com salt aleatorio antes de persistir. Na autenticacao, o BCrypt compara o hash — nunca a senha original.

Geracao do Token JWT:

```
SecretKey chave = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8)); String
token = Jwts.builder().subject(email).issuedAt(new Date()).expiration(new
Date(System.currentTimeMillis() + expiration)).signWith(chave).compact();
```

Flyway — Controle de Schema

O Flyway garante que o banco seja criado e atualizado de forma controlada. Cada migracao e um arquivo SQL versionado (V1__, V2__...) que so executa uma vez. Em producao, isso e essencial — nunca usamos ddl-auto: update que pode apagar dados.

Soft Delete

Em sistemas financeiros NUNCA se apaga um registro permanentemente. Isso e requisito de auditoria. A exclusao logica marca o campo 'ativo = false'. Todos os endpoints de listagem filtram por WHERE ativo = true automaticamente. Pergunta de entrevista: 'Como voce implementou exclusao de dados?' Resposta: Soft delete com campo booleano 'ativo'. Jamais DELETE fisico em dados financeiros.

4. ms-centro-custo (porta 8082)

Gerencia centros de custo — categorias organizacionais para agrupar lançamentos. Cada centro de custo pertence a um usuario especifico (isolamento por usuariold extraido do JWT).

Funcionalidades

- POST /centros-custo — cria novo centro de custo
- GET /centros-custo — lista centros do usuario autenticado
- GET /centros-custo/{id} — busca por ID (valida que pertence ao usuario)
- PUT /centros-custo/{id} — atualiza nome/descricao
- DELETE /centros-custo/{id} — soft delete

Isolamento por usuario

O usuariold nao vem no corpo da requisicao. E extraido automaticamente do JWT via `SecurityContextHolder.getContext().getAuthentication().getName()`. Isso garante que um usuario jamais acesse ou modifique dados de outro usuario.

5. ms-lancamentos (porta 8083)

O microservico mais complexo do sistema. Gerencia todos os lancamentos financeiros com maquina de estados, campos de juros/desconto, integracao com ms-centro-custo via Feign e 6 migracoes Flyway.

Campos do Lancamento

Campo	Tipo	Descricao
id	Long	Identificador unico
descricao	String	Descricao do lancamento
tipo	Enum	RECEITA ou DESPESA
status	Enum	PENDENTE, EFETIVADO ou CANCELADO
valor	BigDecimal	Valor final (valorOriginal + juros - desconto)
valorOriginal	BigDecimal	Valor antes de juros/desconto
taxaJuros	BigDecimal	Percentual de juros
tipoJuros	Enum	SIMPLES ou COMPOSTO
juros	BigDecimal	Valor calculado dos juros
desconto	BigDecimal	Valor de desconto
formaPagamento	Enum	PIX, BOLETO, CARTAO_CREDITO, etc
dataLancamento	LocalDate	Data de criacao
dataVencimento	LocalDate	Data de vencimento
dataPagamento	LocalDate	Data de efetivacao
centroCustold	Long	FK para ms-centro-custo (validado via Feign)
usuariold	String	Email extraido do JWT (nunca vem no body)
ativo	Boolean	Soft delete

Maquina de Estados

PENDENTE -> EFETIVADO (PATCH /lancamentos/{id}/efetivar) PENDENTE -> CANCELADO (PATCH /lancamentos/{id}/cancelar) Por que PATCH e nao PUT? PATCH atualiza parcialmente (so o status). PUT substituiria o objeto inteiro. Para mudanca de status, PATCH e semanticamente correto.

Integracao com ms-centro-custo via Feign

Quando um lancamento e criado com centroCustold, o ms-lancamentos chama o ms-centro-custo via Feign para validar que o centro existe e pertence ao usuario. Se nao existir, retorna 404 antes de salvar o lancamento.

```
@FeignClient(name = "ms-centro-custo", url = "${ms-centro-custo.url}") public interface
CentroCustoClient { @GetMapping("/centros-custo/{id}") CentroCustoResponseDTO
buscarPorId(@PathVariable Long id, @RequestHeader("Authorization") String token); }
```

Por que BigDecimal e nao Double?

Double usa ponto flutuante binario (IEEE 754) e tem imprecisao numerica: $0.1 + 0.2 = 0.30000000000000004$ Em sistemas financeiros isso e INACEITAVEL. R\$0,30 e diferente de R\$0,30000000000000004. BigDecimal usa aritmetica decimal exata — obrigatorio para qualquer calculo monetario.

6. ms-fluxo-caixa (porta 8084)

Servico de calculo sem banco de dados proprio. Consome o ms-lancamentos via Feign, filtra os lancamentos EFETIVADOS e calcula totalEntradas, totalSaidas e saldo.

Logica de Calculo

```
BigDecimal totalEntradas = lancamentos.stream() .filter(l ->
"EFETIVADO".equals(l.status()) && "RECEITA".equals(l.tipo()))
.map(LancamentoResponseDTO::valor) .reduce(BigDecimal.ZERO, BigDecimal::add); BigDecimal
totalSaidas = lancamentos.stream() .filter(l -> "EFETIVADO".equals(l.status()) &&
"DESPESA".equals(l.tipo())) .map(LancamentoResponseDTO::valor) .reduce(BigDecimal.ZERO,
BigDecimal::add); BigDecimal saldo = totalEntradas.subtract(totalSaidas);
```

Por que so lancamentos EFETIVADOS?

Lancamentos PENDENTES sao compromissos futuros — ainda nao aconteceram financeiramente. CANCELADOS foram descartados. O saldo real do caixa so considera o que efetivamente entrou ou saiu.

Repasse do Token JWT

O ms-fluxo-caixa recebe o token JWT do cliente no header Authorization e o repassa para o ms-lancamentos via @RequestHeader. Isso garante que o ms-lancamentos valide o token e retorne so os lancamentos do usuario correto.

7. api-gateway (porta 8080)

Ponto unico de entrada do sistema. Todas as requisicoes do cliente passam pelo gateway, que valida o JWT e roteia para o microservico correto. Construido com Spring Cloud Gateway (WebFlux/reactivo) — diferente dos outros servicos que usam Spring MVC (Servlet).

Rotas Configuradas

Rota	Destino	JWT Obrigatorio?
/auth/**	ms-usuarios:8081	NAO (login/registro)
/usuarios/**	ms-usuarios:8081	SIM
/lancamentos/**	ms-lancamentos:8083	SIM
/centros-custo/**	ms-centro-custo:8082	SIM
/fluxo-caixa/**	ms-fluxo-caixa:8084	SIM

WebFlux vs Spring MVC

O api-gateway usa Spring Cloud Gateway que e baseado em WebFlux (reativo/nao-bloqueante). Por isso, algumas classes sao diferentes: • `GlobalFilter + Ordered` (nao `OncePerRequestFilter`) • `ServerHttpSecurity` (nao `HttpSecurity`) • `@EnableWebFluxSecurity` (nao `@EnableWebSecurity`) • `ServerWebExchange` (nao `HttpServletRequest`) Isso e importante: misturar classes MVC com WebFlux causa erros de compilacao.

JwtGatewayFilter

O filtro intercepta todas as requisicoes antes de chegarem aos microservicos. Rotas publicas (/auth/login, /auth/regar) passam sem validacao. As demais exigem header Authorization: Bearer {token} valido.

8. bff-financeiro (porta 8085)

Implementa o padrao BFF (Backend for Frontend). Em vez do frontend fazer 3 chamadas separadas (lancamentos, centros-custo, fluxo-caixa), o BFF agrega tudo em uma unica resposta. Isso reduz latencia, simplifica o codigo do frontend e melhora a experiencia.

Padrao BFF — Por que usar?

Sem BFF: frontend faz 3 requisicoes -> 3 round trips -> 3 tratamentos de loading/erro Com BFF: frontend faz 1 requisicao -> 1 round trip -> 1 tratamento de loading/erro O BFF e especialmente util quando o frontend precisa de dados de multiplos servicos para renderizar UMA tela. Ex: tela principal do app financeiro precisa do saldo, lista de lancamentos e centros de custo simultaneamente.

Endpoint e Resposta

```
GET /dashboard (Authorization: Bearer {token}) { "fluxoCaixa": { "totalEntradas": 5000.00, "totalSaidas": 1650.00, "saldo": 3350.00 }, "lancamentos": [ {...}, {...} ], "centrosCusto": [ {...}, {...} ] }
```

Clientes Feign no BFF

- LancamentoClient -> GET http://ms-lancamentos:8083/lancamentos
- CentroCustoClient -> GET http://ms-centro-custo:8082/centros-custo
- FluxoCaixaClient -> GET http://ms-fluxo-caixa:8084/fluxo-caixa

9. Seguranca e JWT

Como o JWT funciona neste sistema

O JSON Web Token (JWT) e um padrao aberto (RFC 7519) para transmitir informacoes de forma segura entre partes como um objeto JSON assinado digitalmente.

Etapa	Descricao
1. Login	Cliente envia email+senha para POST /auth/login
2. Autenticacao	ms-usuarios valida credenciais com BCrypt
3. Emissao	ms-usuarios gera JWT assinado com JWT_SECRET e retorna ao cliente
4. Uso	Cliente envia JWT no header: Authorization: Bearer {token}
5. Validacao	Cada microservico valida o JWT com o mesmo JWT_SECRET
6. Extracao	Email do usuario e extraido do 'subject' do token

JWT independente em cada microservico

Pergunta: 'Por que cada microservico valida o JWT em vez de centralizar?' Resposta: Autonomia e resiliencia. Se o ms-usuarios cair, os outros servicos continuam validando tokens existentes porque conhecem o JWT_SECRET. O segredo e compartilhado via variavel de ambiente — identico em todos os servicos. Isso e o padrao JWT stateless: nao ha sessao centralizada.

401 vs 403

401 Unauthorized: O cliente nao provou quem e. Token ausente, invalido ou expirado. Tambem usado para email/senha incorretos (nao autenticado). **403 Forbidden:** O cliente e conhecido mas nao tem permissao para o recurso. Ex: usuario comum tentando acessar rota de admin.

10. Docker e Infraestrutura

O projeto usa Docker Compose para orquestrar todos os containers. Com um unico comando (docker-compose up --build), todo o ambiente sobe: 3 bancos PostgreSQL e 6 microservicos.

Estrutura dos Containers

Container	Imagem	Porta
postgres-usuarios	postgres:15-alpine	5433:5432
postgres-centrocosto	postgres:15-alpine	5434:5432
postgres-lancamentos	postgres:15-alpine	5435:5432
ms-usuarios	build local	8081:8081
ms-centro-custo	build local	8082:8082
ms-lancamentos	build local	8083:8083
ms-fluxo-caixa	build local	8084:8084
bff-financeiro	build local	8085:8085
api-gateway	build local	8080:8080

Dockerfile — Multi-stage Build

```
FROM maven:3.9.6-eclipse-temurin-17 AS build WORKDIR /app COPY pom.xml . RUN mvn dependency:go-offline -B COPY src ./src RUN mvn package -DskipTests FROM eclipse-temurin:17-jre-alpine WORKDIR /app COPY --from=build /app/target/*.jar app.jar EXPOSE 8081 ENTRYPOINT ["java", "-jar", "app.jar"]
```

O multi-stage build usa duas imagens: uma para compilar (Maven completo, ~600MB) e outra so para executar (JRE Alpine, ~100MB). A imagem final e muito menor porque nao inclui o Maven nem o codigo fonte.

Comunicacao entre Containers

Dentro do Docker, containers se comunicam pelo nome do container (nao localhost). Local: ms-lancamentos.url=http://localhost:8083 Docker: MS_LANCAMENTOS_URL=http://ms-lancamentos:8083 A solucao: application.yaml usa valor padrao local com override via env var: ms-lancamentos.url: \${MS_LANCAMENTOS_URL:http://localhost:8083} Assim o mesmo jar funciona localmente e no Docker sem nenhuma alteracao.

11. CI/CD com GitHub Actions

Cada microserviço tem um pipeline de CI configurado no GitHub Actions. A cada Pull Request, o pipeline executa automaticamente os testes, garantindo que nenhum código quebrado entre na branch main.

Pipeline configurado

```
name: CI - ms-lancamentos on: push: branches: [main, developer, feature/**] pull_request: branches: [main, developer] jobs: build: runs-on: ubuntu-latest services: postgres: image: postgres:15 env: POSTGRES_DB: CF_lancamentos POSTGRES_PASSWORD: test steps: - uses: actions/checkout@v3 - uses: actions/setup-java@v3 - run: mvn test
```

@TestPropertySource — Segredo dos testes no CI

O CI não tem acesso às variáveis de ambiente do IntelliJ (JWT_SECRET, etc). A solução é a anotação @TestPropertySource que injeta as propriedades diretamente na classe de teste, sem depender do application.yaml.

```
@SpringBootTest @TestPropertySource(properties = { "jwt.secret=minha-chave-secreta-super-segura-financeiro-2026", "spring.cloud.compatibility-verifier.enabled=false", "ms-lancamentos.url=http://localhost:8083" }) class MsLancamentosApplicationTests { @Test void contextLoads() {} }
```

12. Perguntas e Respostas para Entrevista

Esta secao contem as perguntas mais comuns em entrevistas tecnicas para vagas de backend Java no Mercado Livre, com respostas baseadas neste projeto.

P: Me explique a arquitetura do seu projeto.

R: O sistema e dividido em 6 microservicos Spring Boot independentes. Cada um tem sua propria responsabilidade e banco de dados. O api-gateway centraliza o acesso e valida o JWT antes de rotear as requisicoes. O bff-financeiro agrega dados de 3 servicos numa unica chamada para o frontend. Os servicos se comunicam via HTTP usando Spring Cloud OpenFeign.

P: Por que voce escolheu microservicos e nao monolito?

R: Escalabilidade independente: posso escalar so o ms-lancamentos se estiver sobrecarregado. Isolamento de falhas: se o ms-centro-custo cair, login e lancamentos continuam funcionando. Deploy independente: cada servico tem seu proprio CI/CD. Para um portfolio, tambem demonstra dominio de tecnologias que o Mercado Livre usa em producao.

P: Como voce implementou a autenticacao?

R: JWT estatico com JJWT 0.12.6. O ms-usuarios emite o token assinado com HMAC-SHA (HS384) usando uma chave secreta compartilhada via variavel de ambiente. Cada microservico valida o token localmente com a mesma chave — sem chamada ao ms-usuarios. Isso e stateless: nao ha sessao ou banco de tokens.

P: O que e Spring Cloud OpenFeign e por que voce usou?

R: E uma biblioteca que transforma interfaces Java em clientes HTTP REST. Em vez de escrever RestTemplate ou HttpClient manualmente, declaro a interface com @FeignClient e o Spring cria toda a implementacao. Usei para o ms-lancamentos chamar o ms-centro-custo, para o ms-fluxo-caixa chamar o ms-lancamentos, e no bff-financeiro para chamar os 3 servicos.

P: O que e o padrao BFF?

R: Backend for Frontend e um padrao onde criamos um backend especifico para as necessidades do frontend. Em vez do frontend fazer 3 chamadas para buscar lancamentos, centros-custo e fluxo-caixa, o bff-financeiro agrega tudo num unico endpoint GET /dashboard. Isso reduz round trips, simplifica o codigo do cliente e melhora a performance.

P: Por que BigDecimal e nao Double para valores monetarios?

R: Double usa ponto flutuante binario (IEEE 754) que nao representa todos os decimais com precisao: $0.1 + 0.2 = 0.30000000000000004$ em Java. Em sistemas financeiros, essa imprecisao e inaceitavel — pode resultar em centavos a mais ou a menos. BigDecimal usa aritmetica decimal exata e e o padrao da industria para dinheiro.

P: Como voce implementou soft delete?

R: Todo registro tem um campo boolean 'ativo'. O DELETE nao apaga o registro — apenas marca ativo=false. Todos os endpoints de listagem filtram automaticamente por ativo=true. Em sistemas

financeiros isso e requisito de auditoria: dados financeiros jamais devem ser apagados permanentemente.

P: O que e o Flyway e por que nao usar ddl-auto: update?

R: Flyway e uma ferramenta de versionamento de banco de dados. Cada alteracao no schema e um arquivo SQL versionado (V1__, V2__...) que executa uma unica vez. ddl-auto: update e imprevisivel em producao — pode apagar colunas ou causar comportamentos inesperados. Flyway garante rastreabilidade total, como um Git para o banco.

P: Qual a diferenca entre 401 e 403?

R: 401 Unauthorized significa que o cliente nao provou quem e: token ausente, invalido ou expirado. Tambem usamos 401 para email/senha incorretos. 403 Forbidden significa que o cliente e conhecido (autenticado) mas nao tem permissao para o recurso solicitado. Ex: usuario comum tentando acessar endpoint de admin.

P: Por que voce usa a mesma mensagem de erro para email inexistente e senha errada?

R: Para evitar user enumeration — um atacante nao deve conseguir descobrir quais emails estao cadastrados. Se retornarmos 'email nao encontrado', o atacante sabe que aquele email nao existe e pode tentar outros. Com a mensagem generica 'E-mail ou senha invalidos', nao revelamos informacao sobre o cadastro.

P: Como o Docker Compose ajuda no desenvolvimento?

R: Com um unico comando (docker-compose up --build), todo o ambiente sobe: 3 bancos PostgreSQL e 6 microservicos. Elimina o problema de 'funciona na minha maquina'. Cada servico usa variaveis de ambiente para configurar URLs internas (http://ms-lancamentos:8083 em vez de localhost:8083).

P: Como voce passa o token JWT entre microservicos?

R: O cliente envia o JWT no header Authorization. Quando um microservico precisa chamar outro (ex: ms-lancamentos chamando ms-centro-custo), ele extrai o token do HttpServletRequest e o passa como @RequestHeader no FeignClient. Assim cada servico na cadeia valida que o usuario tem acesso.

P: O que e Spring Cloud Gateway e como ele funciona?

R: E um proxy reverso e API Gateway baseado em WebFlux (reativo). Centraliza o acesso a todos os microservicos. Configuramos rotas que mapeiam paths para servicos (ex: /lancamentos/** -> ms-lancamentos:8083). Um GlobalFilter intercepta todas as requisicoes para validar o JWT antes de rotear.

P: Qual a diferenca entre WebFlux e Spring MVC?

R: Spring MVC e sincrono e bloqueante: cada thread fica presa ate a resposta chegar. WebFlux e reativo e nao-bloqueante: uma thread pode processar multiplas requisicoes simultaneamente enquanto espera I/O. O api-gateway usa WebFlux porque o Spring Cloud Gateway exige. Os demais servicos usam Spring MVC (mais simples para CRUD tradicional).

P: Como voce configurou o CI/CD?

R: GitHub Actions com um pipeline por repositório. A cada push ou PR nas branches main/developer/feature/*, o pipeline executa os testes automaticamente. Para serviços sem banco, so configuro JWT_SECRET como secret no GitHub. Para serviços com banco, configuro um container PostgreSQL no proprio pipeline. O @TestPropertySource injeta as propriedades necessarias nos testes sem depender de variaveis de ambiente externas.

13. Decisoões de Design Importantes

usuarioid nao vem no body

Em todos os microserviços, o `usuarioid` (email) é extraído automaticamente do JWT via `SecurityContextHolder`. Se viesse no body, um usuário malicioso poderia forjar o ID e acessar dados de outro usuário. Extrair do token assinado, essa manipulação é impossível.

@PostConstruct para inicializar a chave JWT

A `SecretKey` do JWT é criada uma única vez no `@PostConstruct` após o Spring injetar o `JWT_SECRET`. Criar a chave a cada validação de token seria desperdício de CPU. O mesmo padrão é usado em todos os 6 microserviços.

Spring Cloud 2024.0.1 + Spring Boot 3.5.x

Incompatibilidade conhecida: Spring Cloud 2024.0.1 não suporta oficialmente Spring Boot 3.5.x. A correção é `spring.cloud.compatibility-verifier.enabled=false` no `application.yaml`. E no `@TestPropertySource` dos testes. Sem isso, o CI falha.

Portas diferentes para os bancos no Docker

Os 3 PostgreSQL usam portas 5433, 5434 e 5435 no host (em vez de 5432) para não conflitar com um PostgreSQL local que o desenvolvedor possa ter instalado.

SessionCreationPolicy.STATELESS

A sessão HTTP é desabilitada em todos os serviços. Cada requisição é independente e deve apresentar o JWT. Isso é essencial para microserviços: não há estado compartilhado entre requisições.

14. Repositorios no GitHub

Microsservico	Repositorio
ms-usuarios	github.com/Taciolb/ms-usuarios (monorepo)
ms-centro-custo	github.com/Taciolb/ms-centro-custo
ms-lancamentos	github.com/Taciolb/ms-lancamentos
ms-fluxo-caixa	github.com/Taciolb/ms-fluxo-caixa
api-gateway	github.com/Taciolb/ms-api-gateway
bff-financeiro	github.com/Taciolb/bff-financeiro
Infra (Docker)	github.com/Taciolb/controle-financeiro-infra

Dica para a entrevista: Tenha todos os repositorios abertos no browser antes da entrevista. Mostre o codigo real, os commits, o CI passando e o Docker Compose funcionando. Demonstrar que o projeto roda de verdade vale mais que mil palavras.

Boa sorte na entrevista, Tacio! Voce construiu um projeto solido.